

AI CAREER FOR WOMEN

Engineer Spoke

TruthLens — A Multi-Agent AI System for Detecting, Tracing and Countering Online Misinformation

An Agentic AI Ecosystem Against Misinformation

A Project Report

submitted in partial fulfillment of the requirements of
AICW project.

by

Hajra Khan, hajra.252764.cs@mhssce.ac.in

Insha Ansari, insha.252766.cs@mhssce.ac.in

Shifa Shaikh, shifa.252768.cs@mhssce.ac.in

Bushra Kazi, bushra.241813.ci@mhssce.ac.in

Under the Guidance of

Mr. Suraj Chopade

Senior Trainer

M.H. Saboo Siddik College of Engineering



ACKNOWLEDGEMENT

NOTE TO THE TEAM — DELETE THIS BOX BEFORE SUBMISSION. The template states explicitly that the Acknowledgement must be written by the students in their own words and must not be copied. The paragraphs below are a starting skeleton only. Please rewrite them in your own language, and check the pronouns used for the guide before submitting.

We would like to take this opportunity to express our sincere gratitude to all those who supported us, directly or indirectly, during the course of this project work.

We are thankful to our guide, Mr. Suraj Chopade, Senior Trainer, for the guidance and feedback provided throughout the development of TruthLens. The suggestions we received during our review discussions helped us narrow a very broad problem into a system we could actually build, test and defend within the time available to us.

We extend our thanks to M.H. Saboo Siddik College of Engineering and to the AI Career for Women (AICW) programme for providing us with the opportunity, the structure and the resources to undertake this project.

Finally, we thank our families and our classmates for their patience and encouragement during the long debugging sessions that this project demanded of us.

Hajra Khan — Team Lead
Insha Ansari — Member
Shifa Shaikh — Member
Bushra Kazi — Member

ABSTRACT

Online misinformation propagates faster than the manual fact-checking processes designed to correct it. By the time a professional correction is published, the false claim has usually already reached its audience. Existing tools address only fragments of this problem: classifiers label text as true or false but reveal nothing about provenance or future reach, diffusion models assume the false claim has already been identified, and generative approaches frequently fabricate the very citations that a misinformation tool cannot afford to get wrong. This project presents TruthLens, a multi-agent system that compresses the detect-trace-predict-counter cycle into a single automated pipeline. Four specialised agents are coordinated by a LangGraph state machine with conditional routing. A Claim Detector uses a large language model under an anchored severity rubric to extract and score candidate false claims; a semantic resolution stage collapses paraphrases onto canonical claims using sentence-transformer embeddings; an Origin Tracer reconstructs provenance as a Neo4j property graph; a Spread Predictor computes an explainable risk score from five weighted features and reports each feature's numeric contribution; and a Narrative Drafter composes a correction grounded strictly in evidence retrieved from a ChromaDB corpus, returning an explicit no-evidence state rather than fabricating support. High-severity claims are held at a human-in-the-loop gate before publication. The implemented system achieved a measured end-to-end latency of 3.0 seconds on average against an original design target of 90 seconds. Calibrating the detection rubric reduced the proportion of claims escalated for human review from 53 per cent to 15 per cent, and correcting an over-tight retrieval threshold restored evidence grounding that had previously failed on every claim. The work demonstrates that a generative system can be constrained to remain both useful and honest in a domain where confident invention is the principal risk.

Keywords: misinformation detection, multi-agent systems, large language models, retrieval-augmented generation, knowledge graphs, human-in-the-loop, explainable risk scoring.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	9
1.1 Problem Statement	9
1.2 Motivation.....	10
1.3 Objectives	11
1.4 Scope of the Project	12
1.4.1 Within Scope.....	12
1.4.2 Outside Scope	12
CHAPTER 2 LITERATURE SURVEY.....	14
2.1 The Empirical Behaviour of Online Misinformation.....	14
2.2 Automated Detection of False Claims	14
2.3 Modelling Propagation and Spread.....	15
2.4 Retrieval-Augmented Generation and the Hallucination Problem	15
2.5 Multi-Agent Architectures for Language Models.....	16
2.6 Identified Gaps.....	17
2.7 Summary	17
CHAPTER 3 PROPOSED METHODOLOGY	18
3.1 System Design	18
3.1.1 Design Principles	18
3.1.2 Architectural Overview.....	18
3.1.3 Use Case View	19
3.2 Modules Used	20
3.2.1 Ingestion Module	20
3.2.2 Claim Detection Module (Agent 1)	21
3.2.3 Semantic Claim Resolution Module	23

3.2.4	Origin Tracing Module (Agent 2).....	24
3.2.5	Spread Prediction Module (Agent 3)	25
3.2.6	Counter-Narrative Module (Agent 4)	26
3.2.7	Orchestration Module	27
3.2.8	Persistence Layer	28
3.2.9	API and Presentation Layer	30
3.3	Data Flow Diagram.....	31
3.3.1	DFD Level 0 — Context Diagram.....	31
3.3.2	DFD Level 1 — Ingestion and Claim Detection	32
3.3.3	DFD Level 1 — Origin Tracing and Spread Prediction	33
3.3.4	DFD Level 1 — Evidence Retrieval and Counter-Narrative.....	34
3.4	Advantages.....	34
3.5	Requirement Specification.....	35
3.5.1	Hardware Requirements.....	35
3.5.2	Software Requirements	36
CHAPTER 4 IMPLEMENTATION AND RESULT.....		37
4.1	Implementation Overview	37
4.2	Results of Ingestion and Claim Detection	38
4.3	Results of Semantic Claim Resolution	39
4.4	Results of Origin Tracing.....	40
4.5	Results of Spread Prediction.....	41
4.6	Results of Counter-Narrative Generation	41
4.7	Results of Human Review and Publication	43
4.8	Testing, Debugging and Defect Resolution	44
4.9	Consolidated Results.....	45

CHAPTER 5 DISCUSSION AND CONCLUSION	46
5.1 Key Findings	46
5.1.1 Threshold calibration mattered more than model selection	46
5.1.2 Data modelling upstream determined agent capability downstream	46
5.1.3 The capacity to refuse is a functional requirement	46
5.1.4 Explainability was achievable without sacrificing capability	47
5.2 GitHub Link of the Project	47
5.3 Video Recording of Project Demonstration	47
5.4 Limitations	47
5.5 Future Work	48
5.6 Conclusion	49
Appendix A — Configurable Parameters	53
Appendix B — Subject Domain Taxonomy	53
Appendix C — Repository Structure	54

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	System Architecture of the Proposed System	19
Figure 3.2	Use Case Diagram of the Proposed System	20
Figure 3.3	LangGraph Agent Orchestration Workflow	27
Figure 3.4	Neo4j Property-Graph Schema (solid line = relationship, dashed line = property reference)	29
Figure 3.5	Data Flow Diagram — Level 0 (Context Diagram)	31
Figure 3.6	DFD Level 1 — Ingestion and Claim Detection Module	32
Figure 3.7	DFD Level 1 — Origin Tracing and Spread Prediction Modules	33
Figure 3.8	DFD Level 1 — Evidence Retrieval, Counter-Narrative and Review	34
Figure 4.1	Sequence Diagram of a Single Claim Traversing the Pipeline	37
Figure 4.2	Live Claims page showing detected claims with severity, confidence and category	38
Figure 4.3	Manual claim submission through the Inject Signal modal	38
Figure 4.4	Investigation page showing the provenance summary and sighting timeline for a traced claim	40
Figure 4.5	Provenance graph rendered in the Neo4j Aura browser	40
Figure 4.6	Spread Intelligence page showing the risk score and the per-driver contribution breakdown	41
Figure 4.7	Evidence Review page showing retrieved evidence linked to a specific claim	42
Figure 4.8	Counter-Narrative page showing a generated, evidence-grounded correction	43
Figure 4.9	Human review queue with a claim awaiting approval	43
Figure 4.10	Publication confirmation dialog (simulated dispatch)	43
Figure 5.1	Link to the recorded project demonstration	47

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Gaps in existing approaches and the corresponding design response in TruthLens	17
Table 3.1	Ingestion source registry as implemented	21
Table 3.2	Fields returned by the Claim Detection module	22
Table 3.3	Claim routing rules	23
Table 3.4	Calibration of the semantic claim resolution threshold	24
Table 3.5	Features and weights used in the spread risk model	25
Table 3.6	Observed squared-L2 distance bands in the evidence corpus	27
Table 3.7	Node and relationship types in the graph database	29
Table 3.8	REST API endpoints exposed by the backend	30
Table 3.9	Hardware requirements	35
Table 3.10	Software requirements	36
Table 4.1	Effect of detector rubric calibration on review queue load	39
Table 4.2	Representative per-driver decomposition of a computed risk score	41
Table 4.3	Evidence corpus growth following topic-matched seeding	42
Table 4.4	Principal defects identified during development and their resolution	44
Table 4.5	Consolidated measured results	45
Table A.1	Configurable system parameters and their adopted values	53
Table B.1	Subject domain taxonomy shared by claims and evidence	53
Table C.1	Principal directories of the implemented codebase	54

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

Misinformation circulating on public digital platforms now travels considerably faster than the institutions built to correct it. A single misleading assertion about a disease outbreak, an electoral procedure, a natural disaster or a financial market can be republished across news portals, video platforms and messaging applications within minutes of first appearing. Professional fact-checking, by contrast, remains a manual and human-paced activity. An analyst must first notice that a claim exists, then establish whether it has already been debunked elsewhere, then attempt to trace where it originated, then form a judgement about how widely it is likely to travel, and only then research, write and publish a correction. Each of these steps is individually defensible and collectively slow.

The consequence of this asymmetry is that corrections routinely arrive after the audience they were intended to protect has already been reached. The harm caused by a false claim is therefore governed less by the existence of the claim itself than by the interval between its appearance and the arrival of a credible, evidence-backed correction in front of the same readership. Reducing that interval is the intervention with the greatest practical leverage.

The specific gap this project addresses is the absence of an accessible, end-to-end system that performs the whole response cycle as one continuous and auditable process. Existing tools address fragments. Text classifiers assign a truth label to a passage but say nothing about where the passage came from or who is likely to encounter it next. Network-diffusion research models how content propagates but presupposes that the false claim has already been identified and isolated. Fact-checking organisations produce high-quality corrections, but publish them hours or days later, and readers must actively seek them out. No widely available system ingests a live post from a real public source, determines whether it contains a harmful false claim, reconstructs its provenance, estimates its spread risk and drafts an evidence-backed correction as a single pipeline.

A further and more subtle difficulty applies specifically to generative approaches. A large language model asked to rebut a false claim will readily produce a fluent, confident and

entirely fabricated citation in support of its rebuttal. In most application domains this is an inconvenience. In misinformation response it is disqualifying, because a fabricated correction is itself a new piece of misinformation, delivered with institutional authority. Any system built on generative models for this purpose must therefore be architecturally constrained against invention, not merely instructed against it.

1.2 Motivation

Two considerations motivated the selection of this problem. The first was direct and personal. Every member of this team has received forwarded claims that later proved to be false, and has observed how difficult it is for an ordinary reader to verify such a claim quickly, particularly when the claim concerns health or public safety and is circulating within a trusted social circle. The gap between how easy it is to forward a claim and how difficult it is to check one is the practical problem we wanted to work on.

The second consideration was technical. We were interested in agentic artificial intelligence, that is, in systems where several specialised components cooperate on successive stages of a task rather than a single model attempting the entire task alone. Misinformation response is a genuinely staged problem. Detection, provenance reconstruction, spread estimation and rebuttal generation require different information, different reasoning and different failure handling. This makes a multi-agent architecture an honest structural fit for the domain rather than a fashionable framing imposed on it. The project therefore gave us an opportunity to study orchestration, shared state and conditional routing on a problem where those mechanisms are actually warranted.

The potential applications extend beyond the prototype. A system of this shape could support the internal triage queue of a newsroom or a fact-checking organisation, giving analysts a ranked, pre-researched list of claims with provenance and draft corrections already attached. It could provide a public health or disaster-management authority with early warning that a specific harmful claim is gaining traction. It could also serve as an educational instrument, showing readers not merely that a claim is false but where it came from and by what rhetorical techniques it was made persuasive.

1.3 Objectives

The objectives of the project were defined as follows.

1. To build a live ingestion layer that continuously collects public content from real external sources, normalises heterogeneous items into a common post schema, and remains extensible to additional platforms.
2. To implement an autonomous Claim Detector agent that determines whether an ingested post contains a factual claim, whether that claim is likely to be misinformation, and assigns a calibrated severity, a confidence value, a subject category, a stated rationale, the concrete harm at stake and the persuasion techniques employed.
3. To eliminate duplicate reporting of the same underlying assertion through semantic claim resolution, so that multiple paraphrases collapse onto a single canonical claim.
4. To implement an Origin Tracer agent that reconstructs the provenance of each claim as a property graph of claims, sources and reporting relationships, and produces a readable natural-language provenance summary derived strictly from recorded graph facts.
5. To implement a Spread Predictor agent that estimates spread risk and a reproduction-style amplification factor from explainable weighted features, reporting the numeric contribution of every individual driver rather than emitting an opaque score.
6. To implement a Narrative Drafter agent that composes a counter-narrative grounded in evidence retrieved from a curated fact-check corpus, and that explicitly declines to answer when no sufficiently relevant evidence exists.
7. To orchestrate all four agents as a single stateful workflow with conditional routing, so that low-value signals terminate early and only meaningful claims consume downstream computation.
8. To retain a human decision-maker in the loop by routing high-severity, high-confidence claims into a review queue that must be cleared by a person before any counter-narrative is published.
9. To deliver a working web prototype in which every stage of the pipeline is presented against live backend data, with no mock or hard-coded content anywhere in the interface.

1.4 Scope of the Project

1.4.1 Within Scope

The project delivers a functioning end-to-end prototype. Its scope covers live ingestion from three public content sources, four cooperating AI agents, two persistence layers, a REST API and a complete web interface presenting every pipeline stage. The system operates on English-language textual content, including the titles and descriptions associated with video items. Provenance reconstruction, risk scoring, evidence retrieval and counter-narrative drafting are all fully implemented and operational, as is the human review gate and the export path for approved corrections.

1.4.2 Outside Scope

Several boundaries were set deliberately and should be understood as constraints on what the system claims to do.

- **Publication is simulated.** The system prepares, exports and confirms the dispatch of an approved correction, but no content is transmitted to any external platform, regulator or government body. This was a conscious decision: publishing corrections under institutional authority is a governance question, not an engineering one, and is not within the remit of a student project.
- **Closed messaging platforms are not covered.** A substantial proportion of real-world misinformation circulates on private messaging services. These are not accessible through public APIs, and no attempt was made to circumvent that.
- **Only English content is processed.** Multilingual ingestion and detection would be necessary for deployment in the Indian information environment and is identified as future work.
- **Only textual claims are analysed.** Image-based and video-based misinformation is outside the current implementation, although video metadata is ingested.
- **Spread prediction is a calibrated heuristic, not a trained forecast.** No labelled propagation dataset was available to the team, so the risk model is an explainable weighted scheme rather than a statistically validated predictor. This is discussed further in Section 3.2.5 and Chapter 5.
- **The connectors for Reddit, Telegram, Twitter/X and TikTok are scaffolded but not enabled.** Their modules are implemented and registered, but they return no data until the

corresponding API credentials are supplied, and no credentials were obtained during the project period.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews the published work that informs the four functional areas the project touches: the empirical behaviour of misinformation online, automated detection of false claims, modelling of propagation through networks, and the grounding of generative language models against retrieved evidence. It closes by identifying the specific gaps that TruthLens was designed to address.

2.1 The Empirical Behaviour of Online Misinformation

The most widely cited empirical characterisation of the problem is the large-scale study by Vosoughi, Roy and Aral [1], which examined the diffusion of verified true and false news stories on a major social platform. Their central finding is that false information spread significantly farther, faster and more broadly than true information across every category examined, and that this difference was attributable to human sharing behaviour rather than to automated accounts. The result matters directly for system design: it establishes that speed of response is not a secondary convenience but the primary determinant of whether a correction is useful at all, and it justifies building a pipeline optimised for latency rather than for exhaustive analysis.

Broader surveys by Shu et al. [2] and by Zhou and Zafarani [3] organise the field into content-based, social-context-based and propagation-based approaches, and note consistently that these strands are pursued largely in isolation from one another. Guo, Schlichtkrull and Vlachos [4] survey automated fact-checking specifically, decomposing it into claim detection, evidence retrieval and verdict prediction, and observe that most published systems implement only a subset of this pipeline. TruthLens is structured deliberately as an attempt to implement the full sequence rather than one stage of it.

2.2 Automated Detection of False Claims

Early work on automated detection was predominantly supervised and dataset-driven. The LIAR benchmark introduced by Wang [5] provided short political statements with graded truthfulness labels and became a standard evaluation target, while the FEVER dataset of Thorne et al. [6] paired claims with evidence sentences drawn from an encyclopaedic corpus

and required systems to both classify the claim and cite the sentences supporting the decision. The introduction of transformer-based language representations, notably BERT [9], substantially improved performance on both benchmarks.

Two limitations of this paradigm are directly relevant to the present project. First, a model trained on a fixed dataset can only reason about the claims and the world represented in that dataset; novel claims that emerge after the training cut-off are outside its competence. Second, and more importantly for a deployed system, a truth label alone is operationally insufficient. Knowing that a claim is false does not tell an analyst where it started, how urgent it is, who else is carrying it, or what a defensible correction would say. TruthLens therefore treats detection as the first stage of a pipeline rather than as the deliverable, and requires its detector to emit severity, confidence, harm, rhetorical technique and subject category alongside the verdict itself.

2.3 Modelling Propagation and Spread

A parallel line of research models how a claim moves through a network rather than what it says. Monti et al. [10] applied geometric deep learning to propagation graphs and demonstrated that the structure of a diffusion cascade can itself be discriminative of falsity, independent of content. Such approaches build on the graph convolutional formulation of Kipf and Welling [11] and are typically implemented using libraries such as PyTorch Geometric [15].

The decisive constraint here is data. These methods require large corpora of observed cascades with ground-truth labels, which are generally obtained through privileged access to platform data. No such dataset was available to this team. The design decision taken in response, and stated plainly rather than concealed, was to implement spread estimation as an explainable weighted-feature model over signals the system genuinely observes, and to use PyTorch Geometric to assemble the propagation structure rather than to train a predictive network on it. Section 3.2.5 documents this precisely.

2.4 Retrieval-Augmented Generation and the Hallucination Problem

The generative component of any automated correction system is exposed to the failure mode catalogued extensively by Ji et al. [12], whose survey of hallucination in natural language

generation distinguishes between output that is unfaithful to its source and output that is factually incorrect about the world. Both are fatal in this domain.

Retrieval-augmented generation, introduced by Lewis et al. [7], is the established mitigation. Rather than relying on parametric knowledge, the model is supplied at inference time with documents retrieved from an external corpus and is instructed to generate only from them. The retrieval step in such systems is commonly implemented with dense sentence embeddings of the kind introduced by Reimers and Gurevych [8], whose Sentence-BERT architecture produces fixed-length vectors suitable for efficient nearest-neighbour search. TruthLens adopts this pattern directly, using the all-MiniLM-L6-v2 sentence-transformer model for both evidence retrieval and claim deduplication.

A practical observation from the present work, developed in Section 4.2 and Section 4.6, is that the correctness of a retrieval-augmented system depends as heavily on the calibration of its relevance threshold as on the choice of generator. An over-tight threshold produces a system that is technically honest but practically useless, because it refuses to answer even when genuinely relevant evidence is present in the corpus.

2.5 Multi-Agent Architectures for Language Models

The final relevant strand concerns the coordination of multiple language model invocations. The ReAct formulation of Yao et al. [13] established the pattern of interleaving reasoning steps with external actions, and frameworks such as AutoGen [14] generalised this to conversations between multiple specialised agents. The attraction of the approach for the present problem is that each stage of misinformation response requires a different prompt, a different external resource and a different failure response, and attempting to express all of this in a single monolithic prompt produces a system that is neither debuggable nor individually testable.

TruthLens uses a state-machine formulation rather than a conversational one. Agents do not negotiate with one another; they are nodes in a directed graph that read from and write to a shared typed state object, with conditional edges determining which path a given claim takes. This choice trades the flexibility of open-ended agent dialogue for determinism, inspectability and bounded cost, all of which were priorities for a system intended to produce auditable output.

2.6 Identified Gaps

Table 2.1 summarises the limitations identified in the reviewed work and the corresponding design response adopted in this project.

Existing approach	Limitation for operational use	Response in TruthLens
Supervised classifiers trained on fixed benchmarks [5], [6], [9]	Cannot reason about claims emerging after training; a truth label alone does not support action.	Detection performed by a prompted language model under an anchored rubric, emitting severity, confidence, harm, technique and category alongside the verdict.
Propagation and cascade models [10], [11]	Require large labelled cascade corpora obtained through privileged platform access.	Explainable weighted-feature risk model over observed signals; every driver reports its numeric contribution to the score.
Manual fact-checking organisations	High quality but published hours or days later; readers must seek the correction out.	Automated pipeline producing a drafted correction in seconds, with the published fact-checks themselves reused as the retrieval corpus.
Unconstrained generative rebuttal	Fabricates supporting citations, producing authoritative-sounding new misinformation [12].	Strict retrieval grounding; an explicit no-evidence state is returned when no document passes the relevance threshold.
Single-stage or partial pipelines [4]	Detection, provenance and response are pursued in isolation.	Four agents orchestrated as one stateful workflow covering detection through to human-reviewed publication.
Fully autonomous automated response	Removes human judgement from consequential decisions.	Severity-gated human-in-the-loop review; high-severity claims cannot be published without explicit approval.

Table 2.1: Gaps in existing approaches and the corresponding design response in TruthLens

2.7 Summary

The literature establishes that speed determines whether a correction is useful, that detection alone is operationally insufficient, that propagation modelling is constrained by data availability, and that generative rebuttal is unsafe unless architecturally grounded. TruthLens responds to these four findings with, respectively, a low-latency orchestrated pipeline, a detector that emits actionable structured output, an explainable rather than learned risk model, and a generator that is permitted to refuse. The chapters that follow describe how this was implemented and what was observed when it was run.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Design

3.1.1 Design Principles

Four principles governed the design of the system and are visible throughout the implementation.

1. **Separation of stages.** Each stage of the response cycle is an independent agent with its own prompt, its own external resource and its own failure behaviour. This permits each to be tested and calibrated separately.
2. **Grounded output.** No agent is permitted to assert a fact it cannot derive from a recorded source. Provenance statements come from graph queries, risk figures come from arithmetic over observed features, and counter-narratives come from retrieved documents.
3. **Explainability by construction.** Every numeric output is accompanied by the reasoning that produced it. The detector returns a rationale and the verbatim phrases that drove its judgement; the spread model returns the contribution of each feature to the final score.
4. **Early termination.** Signals that do not warrant analysis are dropped immediately after detection, before any expensive downstream work is performed. This is what makes the observed latency achievable under a rate-limited inference budget.

3.1.2 Architectural Overview

The overall architecture of the proposed system is shown in Figure 3.1. The system is organised into four horizontal layers. The **ingestion layer** communicates with external content providers and produces normalised post records. The **agent layer** contains the four analytical agents and the orchestration graph that coordinates them. The **persistence layer** comprises a graph database holding claims, sources, reviews and audit events, and a vector database holding the fact-check evidence corpus and the canonical claim index. The **presentation layer** consists of a REST API, an adapter service and a web client. Data flows downward from ingestion through analysis into persistence, and upward from persistence

through the API into the interface, with the human reviewer acting on the system at the boundary between the agent layer and publication.

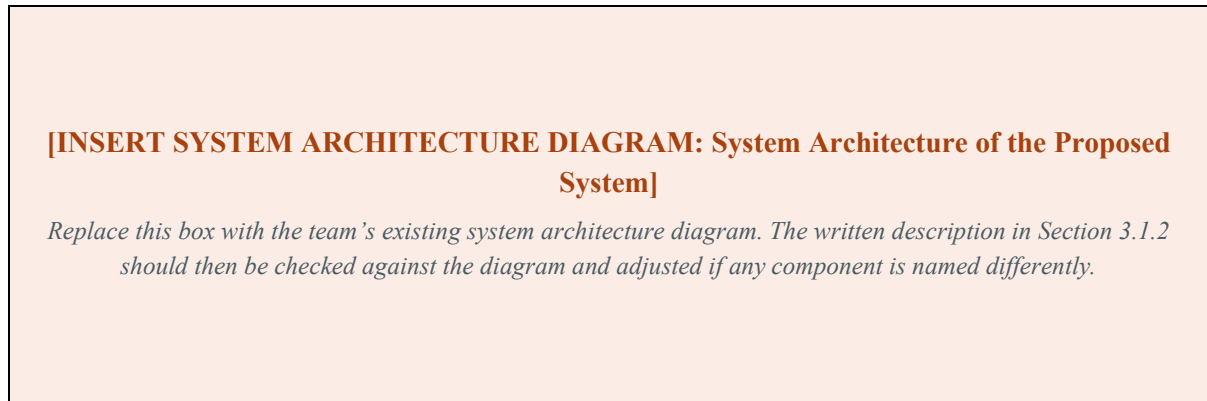


Figure 3.1: System Architecture of the Proposed System

3.1.3 Use Case View

Figure 3.2 presents the use case view of the system. Three actors interact with TruthLens. The **Scheduler / System** actor initiates ingestion and drives the analytical pipeline without human intervention. The **External Content APIs** actor supplies the raw material to be analysed. The **Human Reviewer** actor inspects completed analyses, approves or rejects high-severity claims, and triggers the export of an approved correction. Notably, the reviewer does not participate in detection, tracing or scoring; human attention is deliberately reserved for the decision that carries consequences, namely whether a correction is issued.

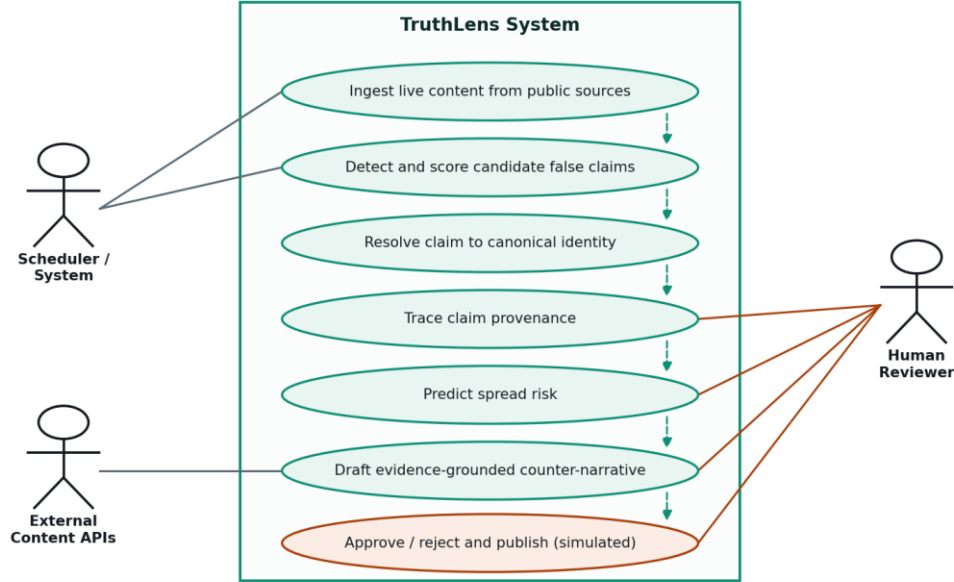


Figure 3.2: Use Case Diagram of the Proposed System

3.2 Modules Used

The system comprises nine implemented modules. Each is described below in the order in which a claim encounters it.

3.2.1 Ingestion Module

The ingestion module is responsible for acquiring content and reducing it to a uniform representation. It maintains a registry of every source the system knows about, recording for each whether it is currently operational and whether its credentials have been supplied. Table 3.1 lists the registry as implemented.

Source	Category	Library / API	Status in this project
NewsAPI	Content	NewsAPI REST	Operational
YouTube Data API v3	Content	Google API client	Operational
RSS Feeds	Content	Feed parser	Operational
Reddit	Social	PRAW	Implemented; awaiting credentials
Telegram	Social	Telethon	Implemented; awaiting credentials and session
X (Twitter)	Social	X API	Implemented; awaiting credentials

TikTok	Social	TikTok Research API	Implemented; awaiting credentials
--------	--------	---------------------	-----------------------------------

Table 3.1: Ingestion source registry as implemented

Each retrieved item is reduced to a common record containing the text, the originating platform, the source name, the author, the URL and the publication timestamp. Three transformations are then applied. First, the text is normalised: typographic punctuation and malformed upstream decodings are folded down to plain ASCII, which prevents encoding artefacts from propagating into the databases and into generated output. Second, exact duplicates are removed using a hash of the normalised text. Third, each remaining post is assigned a heuristic **suspicion score**.

The suspicion score is a cheap, non-model signal computed from the presence of characteristic phrases associated with misinformation, urgency markers, excessive exclamation, and an unusually high proportion of capitalised words. It is important to state precisely what this score is and is not. It is **not** a verdict and is never presented as one. Its sole purpose is to rank the candidate pool so that the expensive language model pipeline is applied first to the posts most likely to warrant it, which is a necessary accommodation to a rate-limited inference budget. The score is subsequently reused as one of the five features in the spread risk model, where it functions as a proxy for the rhetorical intensity of the original posting.

3.2.2 Claim Detection Module (Agent 1)

The Claim Detector submits the normalised post to a large language model under a structured prompt and parses the structured response. The prompt is the most heavily engineered artefact in the system, and its structure reflects two problems encountered during development.

The first problem was over-escalation. An unconstrained prompt caused the model to assign high severity to ordinary factual news reporting, on the reasoning that the subject matter was serious. The prompt therefore opens with an explicit exclusion instruction directing the model to classify neutral reporting, fact-checks and debunks, labelled opinion or satire, and broadly accepted statements as non-misinformation with minimum severity. It states directly that reporting the existence of a false claim is not itself misinformation, which was a specific and recurring misclassification.

The second problem was scale drift. Without fixed reference points, severity values were not comparable between runs. The prompt therefore supplies explicit anchors: a severity of 10 denotes a claim that could cause death or injury if acted upon; 9 denotes serious harm to many people, such as a fabricated cure, a false allegation of electoral fraud or a bank-run rumour; 7 to 8 denotes real but narrower harm; 4 to 6 denotes misleading content with limited harm; 2 to 3 denotes trivial content; and 1 denotes content that is not misinformation at all.

The agent returns the structured fields listed in Table 3.2.

Field	Type	Description
claim	Text	The core assertion as a short declarative statement of at most twelve words, with attribution and framing stripped so that two posts making the same claim yield the same sentence.
is_misinformation	Boolean	Whether the post is judged to contain misinformation.
severity	Integer 1–10	Potential harm, fixed to the rubric anchors above.
confidence	Float 0–1	Certainty that the post is misinformation, judged on the evidence present in the text itself.
category	Enumeration	One of seven subject domains shared with the evidence corpus.
rationale	Text	One sentence naming the rubric anchor chosen and the reason.
harm	Text	The concrete harm if a reader believed the claim.
techniques	List	Up to four manipulation tactics actually present in the post.
keywords	List	Up to five phrases copied verbatim from the post that drove the judgement.
status	Enumeration	Routing decision: dropped, pending or review_required.

Table 3.2: Fields returned by the Claim Detection module

The requirement that the claim be emitted as a short, attribution-free declarative sentence is not cosmetic. It is what makes semantic deduplication viable, because two outlets describing the same assertion in different prose will produce closely similar canonical sentences. This dependency is discussed in Section 4.3.

Routing is then determined by the rules in Table 3.3. The design objective was to drop material only when both the stakes and the certainty are low, since a flat confidence cut-off would silently discard genuinely harmful claims about which the model is merely uncertain.

Condition	Assigned status	Consequence
severity $\geq$ 9 and confidence $\geq$	review_required	Full analysis is performed, then the claim is held at

0.6		the human review gate.
not misinformation and severity ≤ 3	dropped	Pipeline terminates; nothing is persisted.
confidence < 0.4 and severity ≤ 4	dropped	Low stakes combined with low certainty; pipeline terminates.
otherwise	pending	Full analysis is performed; no human escalation required.

Table 3.3: Claim routing rules

A safeguard is applied before routing: if the model has judged the post not to be misinformation, its severity is capped at 3 regardless of the value it emitted. This prevents a post from being escalated on the strength of a severity number the model produced inconsistently with its own verdict.

3.2.3 Semantic Claim Resolution Module

This module addresses a structural defect that only became visible once bulk ingestion was operating. Claims are stored in the graph database keyed on their text. The Claim Detector, however, phrases the same underlying assertion slightly differently depending on the wording of the post it was given. A news article and a video description conveying the same false claim therefore produced two distinct claim nodes, each with a single source. The Origin Tracer consequently had no propagation chain to follow, and the Spread Predictor received an outlet count of one and a velocity of zero. Two downstream agents were effectively disabled by a data modelling problem upstream.

The module maintains a separate vector collection of canonical claim texts. Each newly detected claim is embedded and compared against this collection. If the nearest neighbour lies within a calibrated distance threshold, the existing canonical text is returned and all downstream writes are directed to that single node; the alternative wording is recorded on the node as an alias, and a merge event is written to the audit trail. Otherwise the new claim is registered as canonical.

The threshold was set empirically rather than assumed. Table 3.4 reports the squared-L2 distances measured on the calibration set.

Claim A	Claim B	Distance	Correct decision
vaccines cause autism in children	vaccines are linked to autism in kids	0.13	Merge

5G towers cause cancer	5G radiation causes cancer in humans	0.38	Merge
drinking bleach cures covid	bleach solution cures coronavirus	0.48	Merge
the moon landing was faked	Apollo 11 was staged by NASA	0.57	Merge
5G towers cause cancer	5G masts are linked to cancer spikes	0.58	Merge
residents near the 5G mast report a rise in cancer	5G radiation is causing cancer	0.65	Merge
vaccines cause autism	vaccines cause infertility	0.80	Separate
the moon landing was faked	5G towers cause cancer	1.75	Separate

Table 3.4: Calibration of the semantic claim resolution threshold

The widest true paraphrase observed in live ingestion measured 0.65 and the closest genuinely distinct pair measured 0.80. A threshold of 0.70 sits in the gap between them and was adopted. The value is exposed as a configuration parameter so that it can be retuned if the distribution shifts.

3.2.4 Origin Tracing Module (Agent 2)

The Origin Tracer answers two questions: where did this claim first appear within the material the system has observed, and which outlets have since carried it. It operates in three steps.

First, it records the sighting. When a claim arrives with a genuine source URL from a non-manual platform, the module merges a source node into the graph and creates a reporting relationship from the claim to that source, carrying the publication timestamp and the ingestion timestamp as properties. Hand-typed submissions are deliberately excluded from this step, because a manually entered claim has no verifiable outlet and recording one would corrupt the provenance record.

Second, it queries the graph for all sources associated with the claim and orders them by timestamp, earliest first, with undated sightings sorted to the end. From this ordered list it derives the first observed source, the total outlet count and the full sighting timeline.

Third, it produces a natural-language summary. The ordered outlet chain is supplied to the language model with an instruction to describe, in two to three sentences, where the claim appears to have originated and how it spread across the listed outlets, using only the supplied data and inventing no outlets. The separation of responsibilities here is deliberate and is

central to the project's approach: **the facts come from the graph and only the prose comes from the model**. When fewer than two outlets are on record the model is not invoked at all, and a templated statement is returned instead, since there is no propagation path to narrate.

A confidence value is computed for the trace from the evidence actually available: a base value, an increment if the earliest sighting carries a genuine timestamp, and a further increment scaled to the number of corroborating outlets. Where no provenance edges exist at all, the module falls back to reporting either that the claim was previously ingested without source metadata, or that it has not been seen before.

Interpretation caveat, stated in the system and repeated here: the module reports first-seen order within the material the system has ingested. This is not necessarily the true origin of the claim in the world, and the report does not claim otherwise.

3.2.5 Spread Prediction Module (Agent 3)

The Spread Predictor estimates how far a claim is likely to travel. As explained in Section 2.3, no labelled propagation dataset was available, and the module is therefore explicitly built as a transparent feature-weighted model rather than a trained predictor. Five normalised features are combined using the weights in Table 3.5.

Feature	Weight	Normalisation	Origin of the value
Detector severity	0.30	severity / 10	Claim Detection module
Ingest suspicion score	0.20	clamped to 0–1	Ingestion heuristics
Spread velocity	0.20	outlets per day / 5, capped at 1	Provenance timeline
Outlet count	0.15	outlets / 8, capped at 1	Provenance graph
Detector confidence	0.15	clamped to 0–1	Claim Detection module

Table 3.5: Features and weights used in the spread risk model

The weighted sum is multiplied by a penalty factor, which is unity for claims judged to be misinformation and 0.4 otherwise, and the result is clamped to the unit interval. Velocity is computed from the span of real sighting timestamps; where all sightings arrived within approximately six hours the arrival is treated as a burst and the count is scaled upward accordingly. From the risk score a reproduction-style amplification factor is derived, together with a ballpark estimate of current reach and a six-hour projection under contained and uncontained assumptions.

The module's distinguishing behaviour is its reporting. Rather than returning a bare score, it decomposes the score and reports what each feature contributed, ranked by contribution, in a form such as `Detector severity: 7/10 -> +0.210 of 0.396 risk (53%)`. A feature that contributed nothing is reported as having contributed nothing. Where the non-misinformation penalty has been applied, this too is stated explicitly. The output is therefore inspectable by a reviewer who does not have access to the source code.

PyTorch Geometric is used within this module to assemble the propagation structure as a graph data object: observed outlets are linked as a chain in order of sighting, and projected susceptible clusters, whose number scales with the computed risk, are attached to the most recent observed outlet. The reported reach figure is the node count of this assembled graph. To be unambiguous about what this does and does not represent: **no graph neural network is trained and no learned inference is performed**. The library is used for structural representation so that the reported figure reflects real observed topology rather than a constant.

The audience figures used in the reach estimate are order-of-magnitude constants associated with each platform type. They are ballpark proxies, not measured audience data, and are presented as such throughout.

3.2.6 Counter-Narrative Module (Agent 4)

The Narrative Drafter composes the corrective text. It embeds the canonical claim, retrieves the three nearest documents from the evidence corpus, and discards any whose distance exceeds the relevance threshold. If no document survives this filter, the agent returns a status of no-evidence together with a confidence of zero and generates nothing. Only if at least one document survives is the language model invoked, and then under an instruction to correct the claim using only the supplied evidence, to invent no facts, to avoid exaggeration and to maintain a neutral tone. The identifiers of the documents used are persisted alongside the generated text, which is what allows the interface to display the evidence behind each correction and allows a reviewer to audit it.

The relevance threshold governs the behaviour of this module more than any other parameter. Table 3.6 records the distance bands observed on this corpus and the operational interpretation of each.

Distance band	Interpretation	Retrieval decision
≈ 0.3	Near-verbatim restatement of the claim	Accept
$\approx 0.8 - 1.2$	Same topic and genuinely relevant evidence	Accept
> 1.5	Unrelated material	Reject

Table 3.6: Observed squared-L2 distance bands in the evidence corpus

The threshold was initially set to 0.6, which admitted only the first of these bands. The consequences of that choice, and its correction, are documented in Section 4.6 and constitute one of the principal findings of the project.

3.2.7 Orchestration Module

The orchestration module composes the four agents into a single executable workflow, implemented as a state graph. All agents read from and write to a shared typed state object with optional fields, which accumulates the output of each stage as the claim progresses. The graph structure is shown in Figure 3.3.

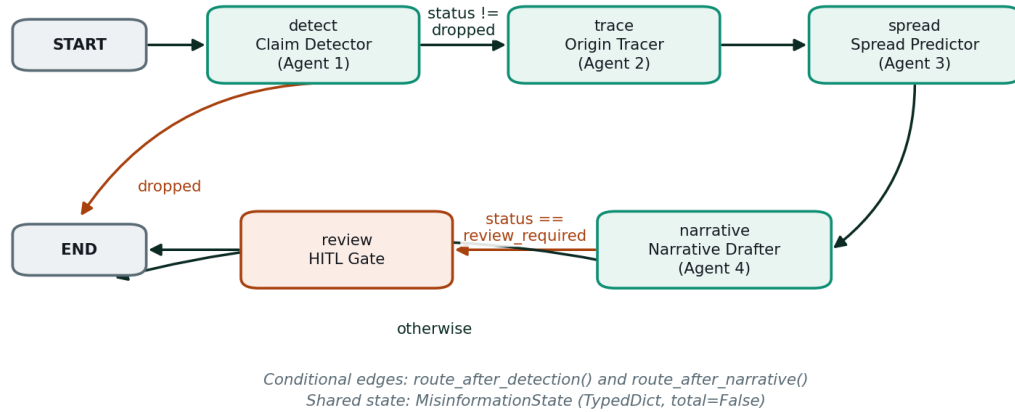


Figure 3.3: LangGraph Agent Orchestration Workflow

Execution begins at the detection node. A conditional edge then routes the claim: if detection returned a status of dropped, the workflow terminates immediately; otherwise it proceeds to tracing, then unconditionally to spread prediction, then unconditionally to narrative drafting.

A second conditional edge after drafting routes claims marked for review to the human review node, and all others to termination.

The ordering here reflects a deliberate revision made during development. In the initial design, claims marked for review were diverted to the human gate immediately after detection. This proved to be the wrong order, because it presented the reviewer with a bare severity score and no provenance, no risk assessment and no draft correction, which is precisely the information a reviewer needs in order to decide. The workflow was restructured so that the complete analysis is always performed first and the human gate acts on a finished dossier. The review node therefore does not discard the drafted narrative; it flags the completed analysis as pending approval and records an audit event.

A second revision concerned persistence. The storage condition was originally expressed as a confidence threshold, which allowed claims the detector had already rejected to be written to the database if they happened to carry high confidence. The condition was rewritten to test the detector's routing status directly, so that a dropped claim is never persisted.

3.2.8 Persistence Layer

Two databases are used, each chosen for the access pattern it serves. A property-graph database stores claims and their relationships, because the provenance question is inherently a traversal over connected entities. A vector database stores the evidence corpus and the canonical claim index, because both require nearest-neighbour search over dense embeddings. The graph schema is shown in Figure 3.4 and summarised in Table 3.7.

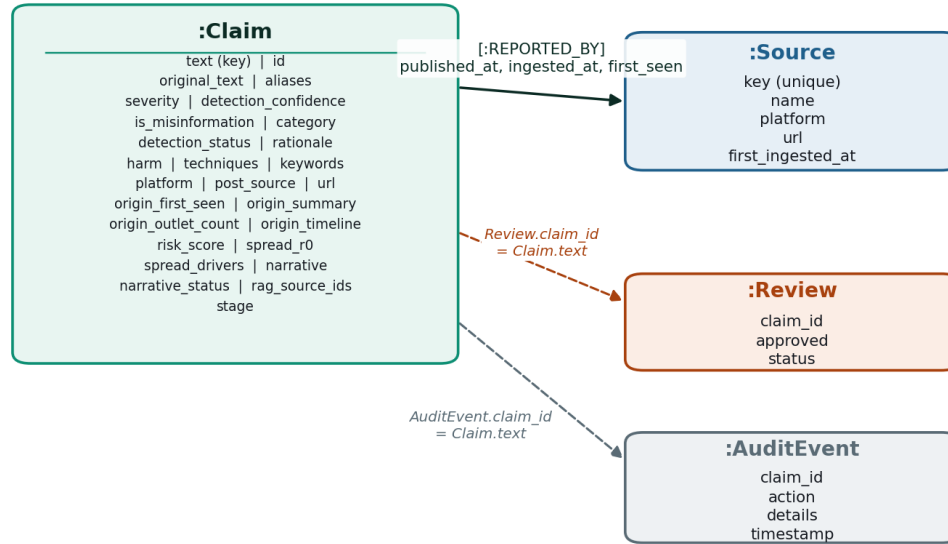


Figure 3.4: Neo4j Property-Graph Schema (solid line = relationship, dashed line = property reference)

Element	Key	Purpose
:Claim	text	The canonical claim, carrying the persisted output of all four agents, the ingestion metadata, the merged aliases and the current pipeline stage.
:Source	key	An outlet that carried the claim, identified by URL where available and otherwise by platform and author.
:Review	claim_id	The human approval decision for a claim.
:AuditEvent	claim_id	An immutable, timestamped record of a pipeline action: detection, merge, trace, spread prediction, drafting, review or drop.
[:REPORTED_BY]	—	Relationship from a claim to a source, carrying the publication, first-seen and ingestion timestamps.

Table 3.7: Node and relationship types in the graph database

The audit event mechanism deserves particular mention. Every consequential action taken by the system on a claim writes an event describing what happened and why, including the severity and confidence at detection, the measured distance when two claims were merged, the risk figures and projections at spread prediction, and the number of evidence documents used at drafting. The complete trail is returned with the claim through the API and is what allows the system's output to be reconstructed and challenged after the fact.

The vector database holds two separate collections. The first is the evidence corpus, seeded from the public feeds of established fact-checking organisations and supplemented with topic-matched articles retrieved for each monitored subject area; each document carries its publisher and a domain label. The second is the canonical claim index described in Section 3.2.3. Keeping them separate is necessary because they answer different questions: one asks what is known to be true, the other asks which claims are already being tracked.

3.2.9 API and Presentation Layer

The backend exposes seventeen REST endpoints, summarised by function in Table 3.8. The full pipeline is available as a single call, and each agent is additionally exposed individually, which was essential during development for testing stages in isolation.

Group	Endpoints	Purpose
Health	/health	Service liveness check.
Ingestion	/api/ingest, /api/ingest/bulk, /api/ingest/bulk/background	Submit a single post, run a bulk ingestion cycle, or run one asynchronously.
Pipeline	/api/workflow/run	Execute the full four-agent workflow on a post.
Individual agents	/api/claims/detect, /api/claims/trace, /api/claims/predict-spread, /api/claims/draft	Invoke a single agent in isolation for testing and debugging.
Claims	/api/claims (GET, POST)	Retrieve all stored claims with review status and audit trail, or create one.
Review	/api/claims/{id}/review (GET, POST)	Read or record the human approval decision.
Publication	/api/claims/{id}/publish	Export an approved counter-narrative (simulated dispatch).
Evidence	/api/evidence (GET, POST)	List the evidence corpus or add a document to it.
Sources	/api/sources	Report which ingestion sources are wired up and configured.

Table 3.8: REST API endpoints exposed by the backend

The web client presents seven functional pages, one for each stage of the pipeline together with a dashboard and a landing page. An adapter service sits between the client and the API and maps the flat response structure into the richer shape the interface consumes. A design decision worth recording is that the client contains **no fallback data**. An earlier version silently substituted sample content when the backend was unreachable, with the result that the interface displayed plausible but entirely fictional figures while the backend was in fact holding different data. All such content was removed and replaced with an explicit offline

indicator, on the principle that a misinformation system displaying invented numbers is self-defeating.

3.3 Data Flow Diagram

A Data Flow Diagram is a graphical representation of the flow of data through an information system, modelling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. The diagrams below decompose TruthLens progressively: Figure 3.5 establishes the system boundary and its external interactions, and Figures 3.6 to 3.8 expand the internal processing into its three functional groups.

3.3.1 DFD Level 0 — Context Diagram

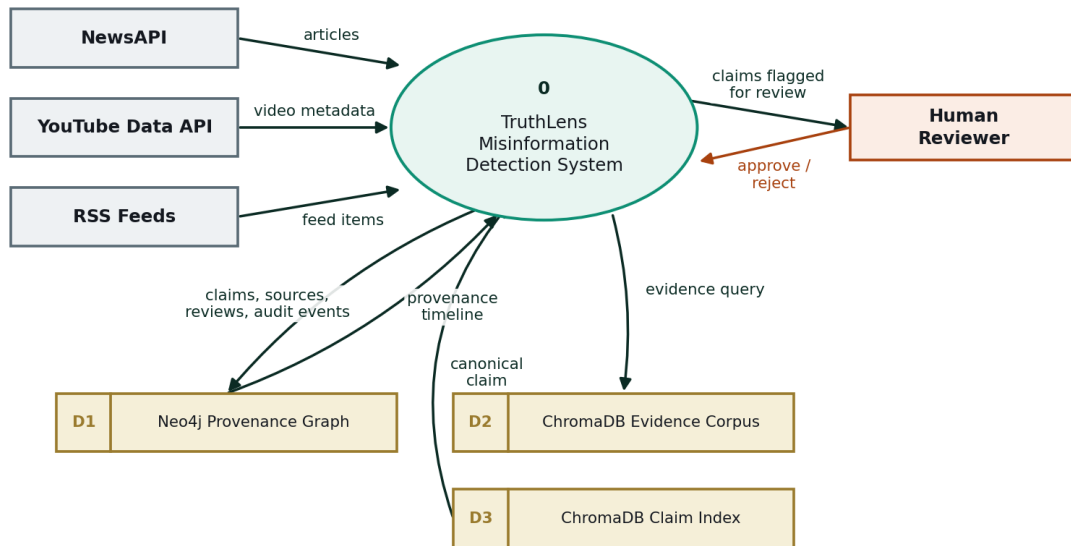


Figure 3.5: Data Flow Diagram — Level 0 (Context Diagram)

At the context level the system has four external interactions. Three content providers supply articles, video metadata and feed items. The human reviewer receives claims flagged for review and returns approval or rejection decisions. Three data stores are maintained internally: the provenance graph, the evidence corpus and the canonical claim index.

3.3.2 DFD Level 1 — Ingestion and Claim Detection

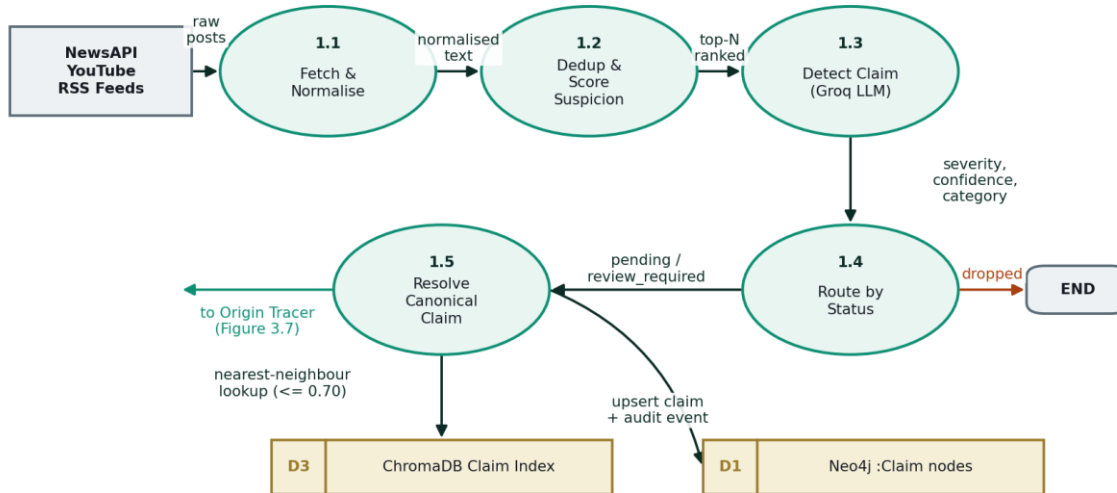


Figure 3.6: DFD Level 1 — Ingestion and Claim Detection Module

Process 1.1 fetches and normalises incoming items. Process 1.2 removes exact duplicates and assigns the heuristic suspicion score, ranking the pool so that the most suspicious posts are processed first. Process 1.3 submits the post to the language model and obtains the structured detection output. Process 1.4 applies the routing rules of Table 3.3, terminating dropped claims immediately. Surviving claims pass to process 1.5, which resolves the claim against the canonical claim index and writes the claim node and its audit event to the graph before handing control to the origin tracing group.

3.3.3 DFD Level 1 — Origin Tracing and Spread Prediction

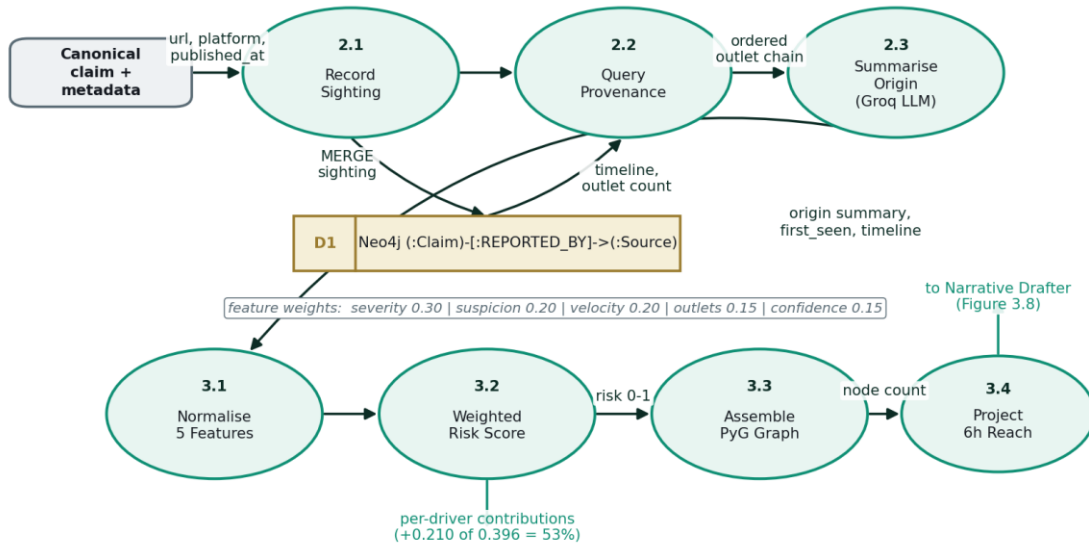


Figure 3.7: DFD Level 1 — Origin Tracing and Spread Prediction Modules

Process 2.1 records the sighting as a relationship in the provenance graph. Process 2.2 reads the graph back to obtain the ordered outlet chain, the first observed source and the outlet count. Process 2.3 converts those facts into a readable summary. The resulting timeline and outlet count then feed the spread group: process 3.1 normalises the five features, process 3.2 computes the weighted risk score and the per-driver contributions, process 3.3 assembles the propagation graph structure, and process 3.4 derives the reach estimate and the six-hour projections.

3.3.4 DFD Level 1 — Evidence Retrieval and Counter-Narrative

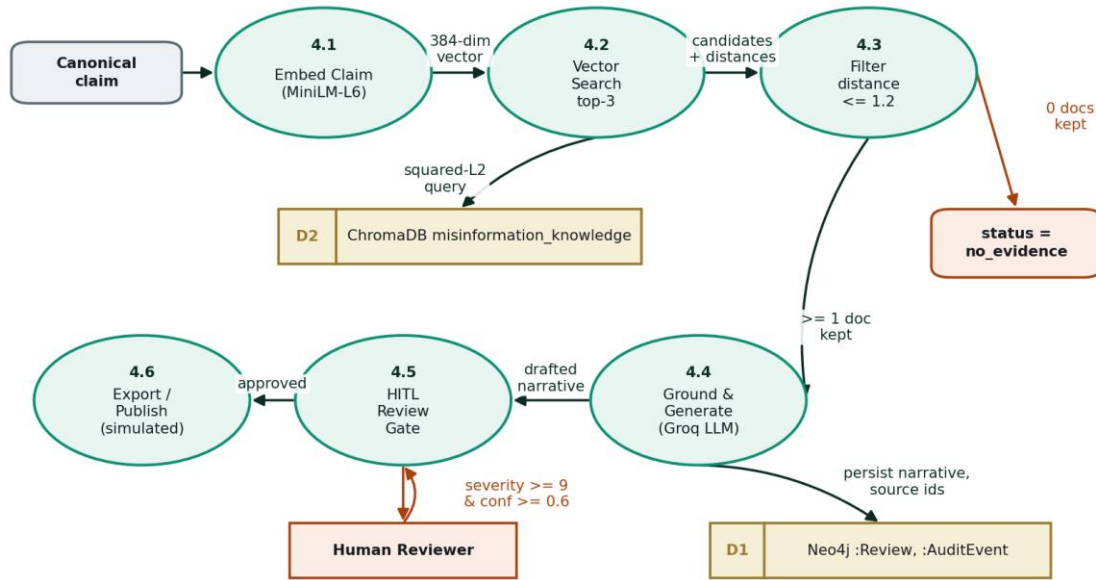


Figure 3.8: DFD Level 1 — Evidence Retrieval, Counter-Narrative and Review

Process 4.1 embeds the canonical claim. Process 4.2 performs the nearest-neighbour query against the evidence corpus. Process 4.3 applies the relevance filter; if no document survives, the pipeline terminates with an explicit no-evidence status and nothing is generated. Otherwise process 4.4 generates the grounded correction and persists it with the identifiers of the documents used. Process 4.5 is the human review gate, which is entered only for claims meeting the severity and confidence condition. Process 4.6 exports an approved correction through the simulated publication path.

3.4 Advantages

The following advantages follow from the design described above.

- **Complete coverage of the response cycle.** Detection, provenance, risk estimation and rebuttal are performed within one pipeline rather than by separate disconnected tools, so a claim emerges as a complete dossier rather than a bare label.
- **Grounded and auditable output.** Provenance statements derive from graph queries, risk figures from arithmetic over observed features, and corrections from retrieved documents

whose identifiers are stored. Every consequential action is recorded as a timestamped audit event.

- **Explainability at every numeric output.** The detector states its rationale and quotes the phrases that drove it; the spread model states what each feature contributed to the score. A reviewer can therefore evaluate the reasoning rather than only the conclusion.
- **A generator that is permitted to refuse.** Returning an explicit no-evidence state rather than an unsupported rebuttal is what makes the output usable in a domain where a fabricated citation would be worse than silence.
- **Efficient use of a constrained inference budget.** Heuristic ranking before detection and early termination after it ensure that expensive model calls are spent on the claims that warrant them.
- **Human judgement preserved where it matters.** Automation handles detection, tracing, scoring and drafting; the decision to publish remains with a person.
- **An evidence base that maintains itself.** The corpus is seeded from live fact-checking feeds and grows as new fact-checks are published, so it does not become stale in the way a static dataset would.
- **Extensibility.** Four additional platform connectors are implemented and registered, and become operational as soon as credentials are supplied, without changes to the pipeline.

3.5 Requirement Specification

3.5.1 Hardware Requirements

Component	Specification
Processor	Intel Core i5 (13th Generation) or equivalent
RAM	16 GB
Storage	512 GB SSD
Graphics	Intel Iris Xe Graphics
Operating System	Windows 11 Pro (64-bit)
Network	Broadband connection (required for all external API calls)

Table 3.9: Hardware requirements

No dedicated graphics processor is required. Language model inference is performed remotely through a hosted API, and the sentence-embedding model used for retrieval and deduplication is small enough to run comfortably on the CPU. The graph database is cloud-hosted; the vector database is embedded and persists to the local filesystem.

3.5.2 Software Requirements

Category	Components
Programming languages	Python 3, TypeScript / JavaScript
Agent orchestration	LangGraph
Language model inference	Groq API serving openai/gpt-oss-20b
Backend framework	FastAPI, Uvicorn
Vector database	ChromaDB (embedded, persistent)
Graph database	Neo4j Aura (cloud-hosted)
Embedding model	Sentence-Transformers all-MiniLM-L6-v2
Deep learning libraries	PyTorch, PyTorch Geometric
Ingestion libraries	Requests, feed parser, PRAW, Telethon, Google API client
Frontend	React 19, Vite, Tailwind CSS v4, Framer Motion, lucide-react
Adapter service	Node.js with Express
External APIs	Groq API, NewsAPI, YouTube Data API v3
Development tools	Visual Studio Code, Git, GitHub, Python virtual environments, npm

Table 3.10: Software requirements

CHAPTER 4

IMPLEMENTATION AND RESULT

4.1 Implementation Overview

The backend comprises thirty Python modules organised into agents, ingestion connectors, storage adapters, graph access and workflow orchestration, together with shared utilities for language model access and text normalisation. The frontend comprises thirty-five TypeScript modules across seven functional pages, a component library and a themed landing page. Figure 4.1 traces the path of a single claim through the implemented system.

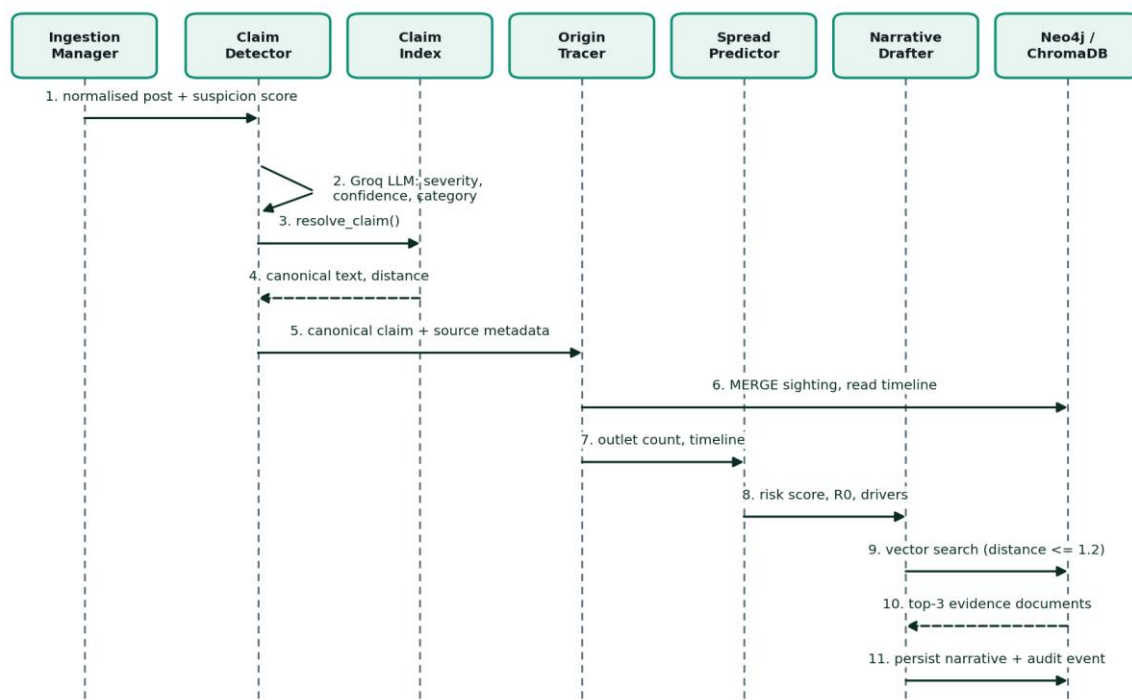


Figure 4.1: Sequence Diagram of a Single Claim Traversing the Pipeline

Two shared utilities proved essential to stable operation and are worth describing before the per-module results.

The first is a shared language model client with rate-limit handling. The inference tier available to the project permits eight thousand tokens per minute. Because a bulk ingestion cycle invokes up to three model calls per claim, a batch of twenty claims reliably exceeded

this allowance and the entire request previously failed with a rate-limit error. The client now parses the retry interval from the provider's own error message, waits accordingly, and falls back to exponential backoff when no interval is given. After four unsuccessful attempts it returns an empty string rather than raising, so that a single failed call degrades one field instead of terminating the batch.

The second is a text normalisation utility applied at ingestion. Content retrieved from external feeds routinely contains typographic punctuation and, in some cases, malformed upstream decodings. These produced visible corruption in stored claims and caused encoding errors when written to the console. Folding such characters to plain ASCII at the point of entry resolved both symptoms at source.

4.2 Results of Ingestion and Claim Detection

Live ingestion operates against the three configured content sources. Retrieved items are normalised, deduplicated by hash, scored for suspicion and ranked, and the highest-ranked subset is passed to the detector.

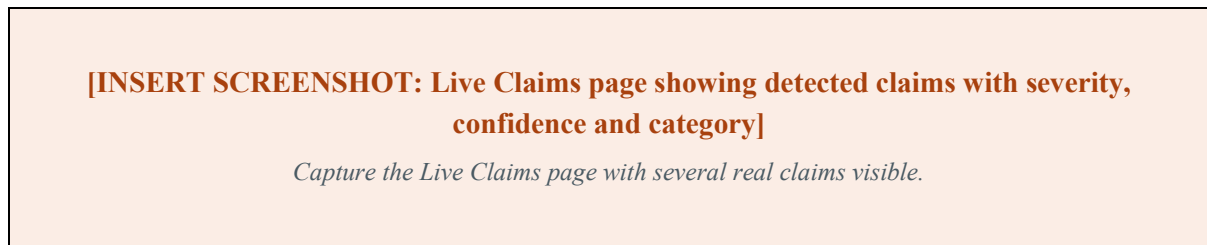


Figure 4.2: Live Claims page showing detected claims with severity, confidence and category

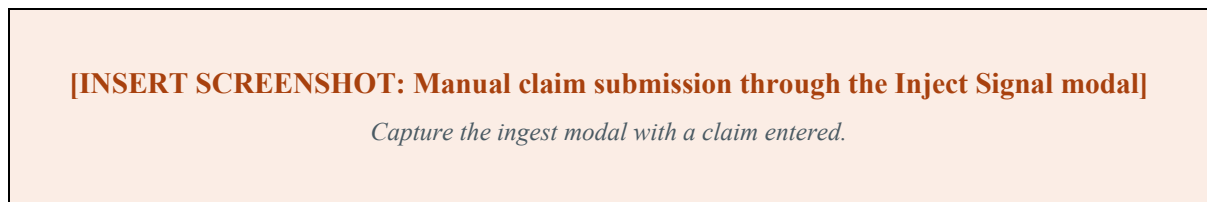


Figure 4.3: Manual claim submission through the Inject Signal modal

The principal measured result for this module concerns the calibration of the detection rubric. In the initial implementation the escalation threshold was set at severity 8 and the prompt contained no exclusion instructions. Under that configuration the detector assigned severity 8 to neutral factual news reporting, and 53 per cent of all analysed claims were routed to human review, which would have made the review queue unusable in practice. Adding the

explicit exclusion instructions, supplying the severity anchors and raising the escalation threshold to 9 reduced this to 15 per cent. Table 4.1 records the change.

Configuration	Prompt structure	Escalation threshold	Claims escalated
Initial	No exclusions, no severity anchors	severity ≥ 8	53%
Calibrated	Explicit exclusions and anchored severity scale	severity ≥ 9 and confidence ≥ 0.6	15%

Table 4.1: Effect of detector rubric calibration on review queue load

It is worth emphasising that no change of model was involved. The improvement was obtained entirely through prompt structure and threshold selection, which suggests that prompt calibration should be treated as a measurable engineering activity with its own acceptance criteria rather than as an informal preliminary.

4.3 Results of Semantic Claim Resolution

The resolution module was evaluated against the eight-pair calibration set reported in Table 3.4, comprising six paraphrase pairs that should merge and two distinct pairs that should remain separate. At the adopted threshold of 0.70 the module produced the correct decision on all eight pairs.

In live operation the module reduced a batch of twenty ingested posts to sixteen canonical claims, consolidating four instances of duplicate reporting. The operational significance of this is larger than the numbers suggest. Before deduplication, each claim node carried a single source, so the Origin Tracer had no chain to follow and the Spread Predictor received an outlet count of one and a velocity of zero for every claim. Deduplication is therefore not a presentational convenience; it is a precondition for two of the four agents producing meaningful output at all.

One diagnostic episode during this work is instructive. Two claims that should have merged were measuring marginally above the threshold, and the initial interpretation was that the threshold required loosening. Inspection of the actual claim texts showed instead that the detector was emitting verbose claims that retained attribution and framing, so the embeddings were separated by the surrounding language rather than by the assertion. The correct remedy was upstream: the detector prompt was amended to require short, attribution-

free declarative assertions. The threshold adjustment from 0.65 to 0.70 was secondary. This illustrates a general point about tuning thresholds before verifying the quality of the representations being compared.

4.4 Results of Origin Tracing

For claims carrying genuine source metadata the module records sightings, reconstructs the ordered outlet timeline and produces a provenance summary derived from the recorded relationships.

[INSERT SCREENSHOT: Investigation page showing the provenance summary and sighting timeline for a traced claim]

Capture the Investigation page for a claim with more than one outlet.

Figure 4.4: Investigation page showing the provenance summary and sighting timeline for a traced claim

[INSERT SCREENSHOT: Provenance graph rendered in the Neo4j Aura browser]

Run a query such as `MATCH (c:Claim)-[r:REPORTED_BY]->(s:Source) RETURN c, r, s LIMIT 25` and capture the resulting graph view.

Figure 4.5: Provenance graph rendered in the Neo4j Aura browser

Behaviour is correct in each of the three defined cases: where provenance edges exist the earliest source, the outlet count and the full timeline are returned with a confidence value scaled to the evidence available; where a claim exists but carries no source metadata the module reports that fact rather than guessing; and where the claim has not been seen before it reports that no prior record exists. Where fewer than two outlets are on record the module returns a templated statement and does not invoke the language model, since there is no propagation to narrate.

One defect was found and corrected during this work. Where a claim had no provenance at all, a placeholder entry was being counted as an outlet, so the interface reported one outlet for claims that in fact had none. The placeholder was given a distinguishable identifier and the displayed count was taken from the length of the recorded timeline instead.

4.5 Results of Spread Prediction

The module produces a risk score, an amplification factor, a current reach estimate, six-hour projections under contained and uncontained assumptions, a measured velocity, and the ranked per-driver contribution list.

[INSERT SCREENSHOT: Spread Intelligence page showing the risk score and the per-driver contribution breakdown]

Capture the Spread page with the driver list clearly legible.

Figure 4.6: Spread Intelligence page showing the risk score and the per-driver contribution breakdown

Table 4.2 shows a representative decomposition produced by the module, illustrating the reporting format rather than a benchmark result.

Driver	Observed value	Contribution	Share of total risk
Detector severity	7 / 10	+0.210	53%
Remaining four features (combined)	—	+0.186	47%
Total risk score	—	0.396	100%

Table 4.2: Representative per-driver decomposition of a computed risk score

The honest characterisation of this module is that it is a transparent, reproducible and inspectable heuristic. It is not a validated forecast, and the report does not present it as one. Its value lies in the fact that a reviewer can see exactly why a claim was ranked as it was and can disagree with a specific weighting rather than with an opaque number. Validating the model against observed outcomes would require a labelled propagation dataset that the project did not possess; this is identified as future work in Section 5.5.

4.6 Results of Counter-Narrative Generation

This module was the subject of the project's most significant defect and its most significant finding.

In the initial implementation the Narrative Drafter returned a no-evidence status for essentially every claim submitted to it. Because the module is deliberately designed to refuse rather than fabricate, this behaviour was not obviously a fault: the system appeared to be correctly declining to answer. Investigation of the retrieval layer showed otherwise. A claim

concerning an asteroid was being matched against a relevant NASA document at a squared-L2 distance of 0.822 — a correct and useful match — and the retrieval filter was discarding it, because the relevance threshold had been set at 0.6. At that value only near-verbatim restatements could pass, and genuinely relevant topical evidence was being rejected in every case. Raising the threshold to 1.2, consistent with the distance bands subsequently measured and reported in Table 3.6, restored retrieval without introducing spurious matches.

The finding generalises beyond this system. A retrieval-augmented architecture can fail silently and in a manner that resembles correct conservative behaviour, and the failure is invisible unless the retrieval distances themselves are inspected. The refusal state that makes the system trustworthy is also what concealed the defect.

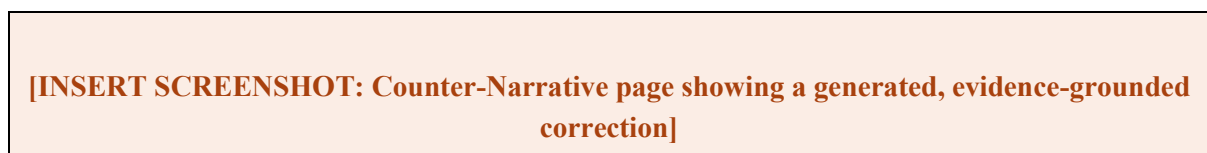
A second, independent cause was identified at the same time. The evidence corpus initially held three documents, harvested from fact-checking feeds without regard to the subjects the system was actually monitoring. Even with a correct threshold there was frequently nothing relevant to retrieve. A topic-matched seeding routine was added, which retrieves material for each monitored subject area in addition to the general fact-check feeds. Table 4.3 records the resulting growth.

Stage	Documents in corpus	Effect on counter-narrative generation
Initial seeding (general fact-check feeds only)	3	Retrieval almost always returned nothing relevant.
After topic-matched seeding	123	Counter-narratives generated successfully; 6 documents linked to active claims.

Table 4.3: Evidence corpus growth following topic-matched seeding



Figure 4.7: Evidence Review page showing retrieved evidence linked to a specific claim



Capture a generated counter-narrative together with its cited sources.

Figure 4.8: Counter-Narrative page showing a generated, evidence-grounded correction

With both corrections in place the module generates grounded corrections for claims with corpus coverage and continues to return the no-evidence state for claims without it. The latter behaviour is retained deliberately: a novel claim for which no fact-check has yet been published will correctly, if unhelpfully, produce no correction, and this is the intended outcome.

4.7 Results of Human Review and Publication

Claims meeting the severity and confidence condition are held at the review gate with their complete analysis attached: detection rationale, provenance summary, risk decomposition and drafted correction. The reviewer's decision is persisted and an audit event is recorded. An approved correction can then be copied, exported or dispatched through the simulated publication path.

[INSERT SCREENSHOT: Human review queue with a claim awaiting approval]

Capture the review queue showing a pending high-severity claim.

Figure 4.9: Human review queue with a claim awaiting approval

[INSERT SCREENSHOT: Publication confirmation dialog (simulated dispatch)]

Capture the confirmation popup shown after publishing.

Figure 4.10: Publication confirmation dialog (simulated dispatch)

Stated explicitly for the avoidance of doubt: the publication step is simulated. The system confirms dispatch within its own interface and records the action in the audit trail, but no content is transmitted to any external platform, regulator or government body.

4.8 Testing, Debugging and Defect Resolution

Testing during this project was conducted primarily through iterative integration testing against live data, supported by the per-agent API endpoints, a system health check script that reports corpus and index sizes, and a standalone detector test harness. The calibration exercises reported in Sections 4.2 and 4.3 constitute the project's structured evaluations. Table 4.4 records the principal defects identified and their resolutions, since the diagnostic path in several cases is more informative than the final figure.

#	Observed symptom	Root cause	Resolution
1	Counter-narrative agent returned no-evidence for every claim.	Retrieval relevance threshold of 0.6 admitted only near-verbatim matches; a correct match at distance 0.822 was being discarded.	Threshold raised to 1.2 after measuring the actual distance bands.
2	Neutral news reporting escalated at severity 8; 53% of claims sent to review.	Detector prompt contained no exclusions and no fixed severity scale.	Explicit exclusions and severity anchors added; escalation threshold raised to 9.
3	Bulk ingestion terminated with a rate-limit error.	Free inference tier permits 8,000 tokens per minute; a 20-claim batch exceeds it.	Shared client added that parses the provider's stated retry interval and backs off.
4	Claims the detector had rejected were appearing in the database.	Persistence was gated on a confidence value rather than on the routing status.	Condition rewritten to test the detector status directly.
5	Origin Tracer reported one outlet for claims with no provenance.	A placeholder node was being counted as a real outlet.	Placeholder given a distinct identifier; count taken from the timeline length.
6	Two paraphrases of the same claim failed to merge.	Detector was emitting verbose claims retaining attribution and framing.	Prompt amended to require short attribution-free assertions; threshold set to 0.70.
7	Interface displayed zero evidence documents while the backend held 123.	The client silently substituted sample data when the backend was unreachable.	All fallback data removed; explicit offline indicator added.
8	Stored claims contained corrupted characters; console writes failed.	Typographic punctuation and malformed upstream decodings in feed content.	Text normalisation applied at ingestion; output encoding set explicitly.
9	Graph database driver failed to initialise at startup.	Connection credentials were present as placeholders rather than values.	Environment configuration completed with real credentials.
10	Frontend could not reach the backend on the local machine.	The host name was resolving to an IPv6 address on which the service was not listening.	Backend address configured explicitly as an IPv4 loopback address.

Table 4.4: Principal defects identified during development and their resolution

4.9 Consolidated Results

Table 4.5 consolidates the measured outcomes of the project. All figures were obtained from the implemented system operating on live data during the development period.

Measure	Result	Basis of measurement
End-to-end pipeline latency	3.0 s average (range 0.9 – 7.5 s)	Full four-agent pipeline on live ingested posts, against an original design target of 90 seconds.
Human review escalation rate	53% → 15%	Proportion of analysed claims routed to review, before and after rubric calibration.
Semantic deduplication accuracy	8 / 8	Correct merge or separate decisions on the calibration set of Table 3.4.
Claim consolidation in live operation	20 posts → 16 claims	Duplicate reporting collapsed onto canonical claims during a bulk ingestion cycle.
Evidence corpus size	3 → 123 documents	Before and after the introduction of topic-matched seeding.
Evidence linked to active claims	6 documents	Retrieved documents bound to specific claims and displayed in the interface.
Backend modules implemented	30	Python modules across agents, ingestion, storage, graph and workflow.
REST endpoints exposed	17	Enumerated in Table 3.8.
Frontend modules implemented	35	TypeScript modules across seven functional pages.

Table 4.5: Consolidated measured results

Scope of evaluation, stated plainly: these figures were obtained on a modest volume of live data over the project period. They are not the result of evaluation against a large, independently labelled benchmark, and should be read as characterising the behaviour of the implemented system rather than as establishing comparative performance against published methods.

CHAPTER 5

DISCUSSION AND CONCLUSION

5.1 Key Findings

Four findings emerged from the implementation that are of wider interest than the prototype itself.

5.1.1 Threshold calibration mattered more than model selection

The single most consequential defect in the project was a retrieval threshold set too tightly, which disabled the counter-narrative agent entirely while producing behaviour that resembled correct conservatism. No change of model, prompt or architecture would have resolved it. Similarly, the reduction in review queue load from 53 per cent to 15 per cent was achieved purely through prompt structure and threshold selection. In both cases the determining factor was the calibration of a numeric boundary against measured data, not the capability of the underlying model. This suggests that in retrieval-augmented and LLM-based systems, threshold selection deserves the same empirical treatment as any other model hyperparameter, and should be documented and justified rather than assumed.

5.1.2 Data modelling upstream determined agent capability downstream

Two of the four agents were producing degenerate output for reasons that had nothing to do with their own logic. Because claims were keyed on exact text, paraphrases of the same assertion became separate nodes, each with a single source; the Origin Tracer therefore had no chain to trace and the Spread Predictor had no outlet count or velocity to work with. Introducing semantic claim resolution restored both. The general lesson is that in a pipeline of cooperating agents, the identity model for the central entity is not a storage detail but a determinant of what the downstream stages are capable of computing.

5.1.3 The capacity to refuse is a functional requirement

Requiring the counter-narrative agent to return an explicit no-evidence state rather than generate an unsupported rebuttal is what makes its output usable. In a domain where a fabricated citation constitutes new misinformation delivered with apparent authority, silence is a better outcome than fluency. The corresponding cost, observed directly in this project, is

that a correctly refusing system and a broken system look identical from the outside, which places an obligation on the developer to instrument the retrieval layer rather than to trust the refusal.

5.1.4 Explainability was achievable without sacrificing capability

Every numeric output in the system is accompanied by its derivation: the detector states its rationale and quotes the phrases that drove it, and the spread model reports the contribution of each feature to the final score. This was obtained through design choices rather than through post-hoc interpretation methods, and it did not require weakening any component. The trade-off was accepted in one place only — the spread model is a transparent heuristic rather than a learned predictor — and that choice was forced by data availability rather than by the explainability requirement.

5.2 GitHub Link of the Project

The complete source code for the project is available at:

<https://github.com/Insha201/TruthLens>

Before submission, confirm that this repository is set to public so that the evaluator can access it, and that no API keys or credentials are present in any tracked file.

5.3 Video Recording of Project Demonstration

A recorded demonstration of the working system accompanies this report.

[INSERT VIDEO LINK: Link to the recorded project demonstration]

Record a walkthrough of the working system and insert the shareable link here. A suggested sequence is: landing page, dashboard, live ingestion, a detected claim with its severity and rationale, the provenance timeline, the spread risk decomposition, the retrieved evidence, the generated counter-narrative, and the human review and simulated publication step.

Figure 5.1: Link to the recorded project demonstration

5.4 Limitations

The following limitations apply to the current implementation and are stated so that the system's output is not over-interpreted.

1. **Spread prediction is a heuristic, not a validated forecast.** The risk score is computed from weighted observed features rather than learned from historical propagation data, because no labelled cascade dataset was available. The reach and projection figures additionally rest on order-of-magnitude audience constants per platform, which are proxies rather than measured audience data.
2. **Provenance is bounded by observation.** The Origin Tracer reports first-seen order within the material the system has ingested. This is not necessarily the true origin of a claim in the world, and a claim that circulated elsewhere before entering the monitored sources will be attributed to the first monitored outlet that carried it.
3. **Source coverage is partial.** Three content sources are operational. The closed messaging platforms on which a substantial proportion of real misinformation circulates are not accessible through public APIs, and four implemented social connectors remain inactive pending credentials.
4. **Counter-narrative quality is bounded by corpus coverage.** A claim for which no relevant fact-check has been published will correctly return the no-evidence state and produce no correction. The system can only rebut what someone has already established.
5. **Publication is simulated.** No correction is transmitted to any external platform or authority.
6. **Only English text is processed.** Multilingual content and image- or video-based claims are outside the current implementation.
7. **Evaluation scope is limited.** Results were obtained on a modest volume of live data over the project period rather than against a large, independently labelled benchmark, and no comparison against published baseline systems was performed.
8. **Detection inherits the characteristics of the underlying model.** The detector's judgements reflect the training and the biases of the language model used, and the anchored rubric constrains but does not eliminate this dependency.

5.5 Future Work

The following extensions follow directly from the limitations above and represent the work we would undertake next.

1. **Activate the remaining connectors.** The Reddit, Telegram, X and TikTok modules are implemented and registered and require only credentials to become operational, which would extend coverage to the platforms where claims propagate fastest.
2. **Replace the spread heuristic with a learned model.** The system already records sighting timelines and outlet counts for every claim it processes. Accumulated over a sufficient period, this becomes the labelled propagation dataset the project lacked, and would permit the weighted scheme to be replaced by a model trained on observed outcomes and validated against them.
3. **Add multilingual ingestion and detection.** This is a prerequisite for any realistic deployment in the Indian information environment and would require both multilingual embeddings and a detector prompt validated in each target language.
4. **Extend analysis to images and video.** A growing proportion of misinformation is not textual, and the ingestion layer already retrieves video metadata that could anchor such an extension.
5. **Evaluate against an established benchmark.** Running the detection stage against a labelled public dataset would permit comparison with published methods and would convert the present behavioural characterisation into a comparative result.
6. **Formalise the publication path.** Establishing an integration with a recognised fact-checking organisation would allow approved corrections to be genuinely published rather than simulated, and would introduce the editorial governance that such a step properly requires.
7. **Introduce automated regression testing.** A fixed set of claims with expected routing decisions and expected retrieval distances would detect calibration drift automatically, given that two of the project's most serious defects were threshold-related and were found only by manual inspection.

5.6 Conclusion

This project set out to determine whether the misinformation response cycle — detect, trace, predict, counter — could be compressed from the hours that manual fact-checking requires into a timeframe short enough to matter, by decomposing the task into specialised cooperating agents rather than asking a single model to perform it alone. The implemented system performs all four stages on live content from real public sources and achieved a

measured average end-to-end latency of 3.0 seconds against an original design target of 90 seconds.

The project's substantive contribution, however, is not that a language model can label a claim as false, which is now routine. It is that a generative system can be architecturally constrained to remain both useful and honest in a domain where confident invention is the principal risk. Every counter-narrative the system produces is bound to retrieved external evidence whose identifiers are stored and displayed. Every risk score exposes the numeric contribution of each of its drivers. Every provenance statement derives from relationships recorded in a graph rather than from model recall. Every consequential action is written to an immutable audit trail. And every high-severity claim is held for human judgement before any correction is issued. Where the system cannot substantiate an answer, it returns no answer rather than an invented one.

The reduction of human review load from 53 per cent of claims to 15 per cent is, in our assessment, as significant as the latency figure. A system that escalates half of everything it sees does not assist a human fact-checker; it merely relocates the workload. Calibrating the system to respect scarce human attention, rather than to consume it, is what makes the human-in-the-loop design viable rather than decorative.

We do not claim that TruthLens solves the problem of online misinformation. Its spread model is a transparent heuristic rather than a validated forecast, its provenance reconstruction is bounded by what it has observed, its coverage excludes the closed platforms where much of the problem resides, and its evaluation was conducted at modest scale. What the project does demonstrate is that the full response cycle can be automated end to end, that it can be made fast enough to be operationally relevant, and that it can be built in such a way that a human being can inspect, challenge and override every conclusion it reaches. For a system intended to counter misinformation, that last property is not an optional refinement. It is the requirement on which the credibility of everything else depends.

REFERENCES

- [1] S. Vosoughi, D. Roy, and S. Aral, “The spread of true and false news online,” *Science*, vol. 359, no. 6380, pp. 1146–1151, 2018.
- [2] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, “Fake news detection on social media: A data mining perspective,” *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017.
- [3] X. Zhou and R. Zafarani, “A survey of fake news: Fundamental theories, detection methods, and opportunities,” *ACM Computing Surveys*, vol. 53, no. 5, 2020.
- [4] Z. Guo, M. Schlichtkrull, and A. Vlachos, “A survey on automated fact-checking,” *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 178–206, 2022.
- [5] W. Y. Wang, “‘Liar, Liar Pants on Fire’: A new benchmark dataset for fake news detection,” in *Proc. 55th Annual Meeting of the Association for Computational Linguistics*, 2017.
- [6] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: A large-scale dataset for fact extraction and VERification,” in *Proc. NAACL-HLT*, 2018.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019.
- [10] F. Monti, F. Frasca, D. Eynard, D. Mannion, and M. M. Bronstein, “Fake news detection on social media using geometric deep learning,” *arXiv preprint arXiv:1902.06673*, 2019.
- [11] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2017.
- [12] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2023.
- [13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [15] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

- [16] LangChain, “LangGraph documentation.” [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [17] Groq, “Groq API documentation.” [Online]. Available: <https://console.groq.com/docs>
- [18] Chroma, “Chroma documentation.” [Online]. Available: <https://docs.trychroma.com/>
- [19] Neo4j, “Neo4j Cypher manual and Aura documentation.” [Online]. Available: <https://neo4j.com/docs/>
- [20] FastAPI, “FastAPI documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [21] Meta Open Source, “React documentation.” [Online]. Available: <https://react.dev/>
- [22] NewsAPI, “NewsAPI documentation.” [Online]. Available: <https://newsapi.org/docs>
- [23] Google Developers, “YouTube Data API v3 documentation.” [Online]. Available: <https://developers.google.com/youtube/v3>
- [24] Snopes Media Group, “Snopes fact-checking.” [Online]. Available: <https://www.snopes.com/>
- [25] Poynter Institute, “PolitiFact.” [Online]. Available: <https://www.politifact.com/>
- [26] Annenberg Public Policy Center, “FactCheck.org.” [Online]. Available: <https://www.factcheck.org/>
- [27] Full Fact, “Full Fact.” [Online]. Available: <https://fullfact.org/>
- [28] World Health Organization, “WHO newsroom and fact sheets.” [Online]. Available: <https://www.who.int/>

APPENDICES

Appendix A — Configurable Parameters

The following parameters govern the behaviour of the system and are exposed as environment configuration so that they can be retuned without modifying source code. The values shown are those adopted for the results reported in Chapter 4.

Parameter	Adopted value	Governs
RAG_MAX_DISTANCE	1.2	Maximum squared-L2 distance at which a retrieved evidence document is accepted as relevant (Section 3.2.6).
CLAIM_DEDUP_MAX_DISTANCE	0.70	Maximum distance at which a newly detected claim is merged onto an existing canonical claim (Section 3.2.3).
REVIEW_SEVERITY_THRESHOLD	9	Minimum severity at which a claim is escalated to human review (Section 3.2.2).
GROQ_MODEL	openai/gpt-oss-20b	The language model used by Agents 1, 2 and 4.
GROQ_MAX_ATTEMPTS	4	Retry attempts before a model call is abandoned (Section 4.1).

Table A.1: Configurable system parameters and their adopted values

Appendix B — Subject Domain Taxonomy

Claims and evidence documents share a single seven-value subject taxonomy, which is what allows the interface to present a claim alongside evidence drawn from the same domain. Claims are assigned a domain by the Claim Detector; evidence documents are assigned one by keyword classification at the time they are added to the corpus.

Domain	Coverage
public_health	Vaccines, disease outbreaks, treatments, medical authorities
elections_civic	Elections, ballots, voting procedures, legislatures, campaigns
emergency_disaster	Earthquakes, floods, storms, fires, evacuations, attacks
financial_panic	Banks, currency, markets, inflation, tariffs, scams
geopolitics	Armed conflict, international relations, sanctions, borders
science_tech	Climate, space, telecommunications, artificial intelligence, energy
other	Claims not falling into any of the above

Table B.1: Subject domain taxonomy shared by claims and evidence

Appendix C — Repository Structure

The implemented codebase is organised as follows. Module counts are as reported in Section 4.1.

Path	Contents
backend/agents/	The four analytical agents.
backend/ingestion/	The ingestion manager and seven platform connectors.
backend/storage/	Vector store adapter, canonical claim index and corpus seeding.
backend/graph/	Graph database client and Cypher operations.
backend/workflow/	The orchestration state graph and shared state definition.
backend/	API surface, shared model client, text normalisation and maintenance scripts.
src/pages/	The seven functional pages of the web client.
src/components/	Shared interface components and visualisations.

Table C.1: Principal directories of the implemented codebase